

Universidad de Sevilla
Máster en Ingeniería de Electrónica, Robótica y Automática (MIERA)

Control en Vehículos

Detección de Carriles y Control de Mantenimiento de Carril

Grupo 1

Agustín Mayor
Rafael Muñoz
José Francisco López

Curso 2025–2026

Índice

1. Introducción	3
2. Detección de carriles	3
2.1. Pipeline de procesamiento de imagen	4
2.2. Cálculo de la posición lateral	5
2.3. Conversión a unidades métricas	5
2.4. Salida del módulo de detección	6
2.5. Pruebas previas sobre vídeo real	6
3. Control de mantenimiento de carril	7
3.1. Modelado del vehículo	7
3.2. Estructura de control en cascada	7
3.3. Sintonización en MATLAB/Simulink	8
3.4. Parámetros del controlador	8
3.5. Transferibilidad de las ganancias	9
4. Integración en simulación y resultados	9
4.1. Entorno de simulación: CARLA Simulator	9
4.2. Arquitectura del sistema	10
4.3. Infraestructura: Docker y contenedores	11
4.4. Nodos ROS2 del sistema	12
4.4.1. Nodo de detección de carriles (<code>lane_detection_node</code>)	12
4.4.2. Nodo de control del vehículo (<code>vehicle_control_node</code>)	12
4.4.3. Nodo generador de anotaciones (<code>annotation_generator_node</code>)	13
4.5. Generación de datasets sintéticos	14
4.6. Resultados	15
5. Conclusiones	15

Resumen

Este trabajo presenta el diseño e implementación de un sistema de *Lane Keeping Assist* (LKA) para un vehículo autónomo simulado en *CARLA Simulator* e integrado con *ROS2 Humble* mediante contenedores Docker. El sistema consta de tres bloques principales: detección de líneas de carril mediante visión clásica con OpenCV y la transformada de Hough, diseño y sintonización de un controlador PID para mantener el vehículo centrado en el carril, y la integración de ambos módulos en un entorno de simulación realista donde se valida el funcionamiento completo. El código fuente, datasets e imágenes y vídeos demostrativos están disponibles en el repositorio público de GitHub: <https://github.com/JoseLopez36/Deteccion-Seguimiento-Carril>.

1. Introducción

Los sistemas de asistencia a la conducción (ADAS) son hoy en día un componente fundamental tanto en la industria del automóvil como en la investigación en robótica móvil. Entre estos sistemas destaca el *Lane Keeping Assist* (LKA), cuyo objetivo es mantener el vehículo centrado en su carril de forma autónoma, corrigiendo continuamente la dirección en función de la posición relativa del vehículo respecto a las marcas viales.

Este trabajo aborda el diseño e implementación de un sistema LKA completo, desde la adquisición de imagen hasta la actuación sobre los mandos del vehículo, validado en el simulador *CARLA Simulator* mediante *ROS2 Humble*. El flujo de funcionamiento del sistema es el siguiente:

1. Adquisición de imágenes desde la cámara RGB frontal del vehículo simulado.
2. Preprocesado de la imagen y detección de las líneas del carril mediante visión clásica.
3. Cálculo del error lateral del vehículo respecto al centro del carril, expresado en metros a partir de los parámetros intrínsecos de la cámara.
4. Generación de los comandos de aceleración/frenado y dirección mediante controladores PID, sintonizados previamente en MATLAB/Simulink.
5. Publicación de los comandos de control al simulador CARLA.

Todo el código fuente, datasets y material multimedia están disponibles en el repositorio público de GitHub del proyecto:

<https://github.com/JoseLopez36/Deteccion-Seguimiento-Carril>

Estructura de la memoria

La memoria se organiza en tres bloques principales:

Sección 2: Detección de carriles. Describe el pipeline de visión artificial empleado para detectar las marcas viales y calcular el error lateral del vehículo.

Sección 3: Control de mantenimiento de carril. Explica el modelado del vehículo y la sintonización del controlador PID en MATLAB/Simulink.

Sección 4: Integración en simulación y resultados. Desarrolla la arquitectura software completa en CARLA/ROS2, la infraestructura Docker, los nodos implementados y los resultados obtenidos.

2. Detección de carriles

La detección de carriles constituye la capa de percepción del sistema LKA, encargada de transformar cada *frame* de la cámara frontal en una estimación numérica de la posición lateral del vehículo dentro de su carril. El módulo se ha implementado mediante técnicas de visión clásica con OpenCV, evitando el uso de redes neuronales para mantener un

pipeline ligero, interpretable y ejecutable en tiempo real sin necesidad de GPU dedicada ni de un dataset de entrenamiento.

2.1. Pipeline de procesamiento de imagen

El detector recibe la imagen RGB de la cámara frontal y aplica una secuencia de etapas clásicas de visión artificial para aislar las marcas viales y ajustar una línea representativa a cada lado del carril:

1. **Realce de color.** La imagen se convierte al espacio de color HLS, en el que se aplican dos umbralizaciones independientes: una para marcas blancas (alta luminosidad) y otra para marcas amarillas (rango de tono específico). La unión de ambas máscaras aísla los píxeles candidatos a pertenecer a una línea de carril, independientemente de su color.
2. **Conversión a escala de grises y suavizado.** La imagen se convierte a escala de grises y se enmascara con el resultado del paso anterior, de forma que únicamente los píxeles de marca vial conservan información. A continuación se aplica un filtro *Gaussian Blur* para atenuar el ruido de alta frecuencia antes de la detección de bordes.
3. **Detección de bordes (Canny).** Se aplica el detector de bordes de Canny sobre la imagen suavizada, con umbrales bajo y alto configurables (`canny_low`, `canny_high`), obteniendo un mapa binario de contornos.
4. **Región de interés (ROI).** Se enmascara la imagen de bordes con un polígono trapezoidal que delimita la zona de carretera frente al vehículo, descartando el cielo, los laterales y otros elementos de la escena ajenos al carril. La altura del horizonte del trapecio es configurable mediante el parámetro `horizon`.
5. **Transformada de Hough probabilística.** Sobre los bordes filtrados por el ROI se ejecuta `cv2.HoughLinesP`, que detecta segmentos de recta caracterizados por su resolución angular y de distancia, el número mínimo de votos (`hough_threshold`), la longitud mínima de segmento (`hough_min_len`) y el hueco máximo tolerado dentro de un mismo segmento (`hough_max_gap`).
6. **Clasificación y ajuste por lado.** Cada segmento detectado se clasifica como perteneciente a la línea izquierda o derecha del carril según el signo de su pendiente, descartando previamente los segmentos casi horizontales mediante el umbral `min_slope`. Los puntos de cada lado se ajustan mediante una regresión lineal (`numpy.polyfit`) que produce una única recta representativa por lado, extrapolada entre la base de la imagen y la altura del horizonte definida por el ROI.
7. **Suavizado temporal.** Las coordenadas de cada línea se promedian con las de los últimos `smoothing_frames` mediante una cola de tipo FIFO, lo que reduce el parpadeo de la detección frame a frame y aporta estabilidad ante oclusiones puntuales o segmentos de Hough espurios.

Todos los parámetros del pipeline se agrupan en una estructura de configuración (`LaneConfig`), lo que permite ajustar el comportamiento del detector –umbrales de

Canny, parámetros de Hough, pendiente mínima, ventana de suavizado y altura del horizonte— sin modificar el código, cargándolos como parámetros ROS2 en tiempo de ejecución.

2.2. Cálculo de la posición lateral

A partir de las líneas izquierda y derecha ajustadas, el detector estima la posición lateral del vehículo dentro del carril evaluando ambas rectas en la fila inferior de la imagen (la más cercana al vehículo). Se distinguen tres casos:

- **Ambas líneas detectadas.** Se calcula el centro del carril como el punto medio entre las coordenadas x de ambas líneas en la base de la imagen, y la anchura del carril como su diferencia. El desplazamiento lateral en píxeles (`offset_px`) se obtiene como la diferencia entre el centro geométrico de la imagen y el centro del carril. Adicionalmente se define una magnitud normalizada `lateral` $\in [0, 1]$, que representa la posición relativa del centro de la imagen respecto a las dos líneas (0 sobre la línea izquierda, 0.5 en el centro exacto del carril, 1 sobre la línea derecha).
- **Solo una línea detectada.** Si únicamente se detecta uno de los dos lados —situación habitual en curvas pronunciadas o marcas viales discontinuas—, el desplazamiento lateral se aproxima respecto a la línea visible, asumiendo que el vehículo se aleja de ella en la dirección del lado no detectado.
- **Ninguna línea detectada.** El estado se marca como `UNKNOWN` y no se actualiza el error lateral, manteniéndose el último valor válido hasta la recuperación de la detección.

A partir del valor normalizado `lateral` se clasifica la posición del vehículo en una de tres zonas discretas, en función de un umbral de centrado configurable (`center_threshold`, fijado en 0.20): `LEFT` si el vehículo está desplazado hacia la línea izquierda más allá del umbral, `RIGHT` si lo está hacia la derecha, y `CENTER` en caso contrario. Esta clasificación por zonas, sumada al valor continuo de `offset_px`, permite tanto generar alertas discretas de cruce o cambio de carril como alimentar un controlador continuo de mantenimiento de carril.

2.3. Conversión a unidades métricas

El error lateral calculado por el pipeline de visión se expresa inicialmente en píxeles, una unidad dependiente de la resolución y la óptica de la cámara. Para emplear dicho error como señal de referencia de un controlador físico es necesario expresarlo en metros, unidad utilizada en el modelo cinemático del vehículo (Sección 3).

La conversión se realiza a partir de la longitud focal f_x de la cámara, obtenida del mensaje `camera_info` publicado por el simulador (elemento K_{00} de la matriz intrínseca). El error lateral en metros se aproxima como:

$$e_{\text{lateral}} [\text{m}] = \frac{\text{offset_px}}{f_x} \quad (1)$$

Esta aproximación asume una proyección de perspectiva simple y es válida bajo la hipótesis de que el plano de la carretera es aproximadamente perpendicular al eje óptico en la región de interés cercana al vehículo. Hasta que el nodo de detección recibe el primer mensaje de `camera_info`, el error se publica directamente en píxeles como valor de respaldo.

2.4. Salida del módulo de detección

El detector expone su resultado mediante una estructura de estado (`LaneState`) que recoge, para cada *frame* procesado: la zona de posición (`CENTER/LEFT/RIGHT/UNKNOWN`), el valor normalizado `lateral`, el desplazamiento `offset_px`, la anchura estimada del carril en píxeles y los indicadores booleanos de detección de cada línea. Esta estructura constituye la interfaz entre el módulo de percepción y el resto del sistema, y es la que consume el nodo `lane_detection_node` (Sección 4) para publicar el error lateral y el estado del carril como mensajes ROS2.

2.5. Pruebas previas sobre vídeo real

Antes de integrar el detector en el simulador CARLA, el pipeline descrito en las secciones anteriores se validó de forma independiente sobre vídeo real de carretera, ejecutando el módulo de detección como aplicación *standalone* en Python sobre secuencias grabadas desde un vehículo. Esta etapa de prueba permitió ajustar los parámetros del pipeline –umbrales de Canny, ventana de suavizado y altura del horizonte del ROI– en un escenario con condiciones de iluminación e infraestructura viaria distintas a las del entorno sintético de CARLA, antes de invertir tiempo en la integración con ROS2 y el simulador.

La Figura 1 muestra un fotograma representativo de estas pruebas, en el que se observa el polígono de carril superpuesto en verde semitransparente, las líneas izquierda (azul) y derecha (amarilla) ajustadas mediante regresión lineal, y el panel de telemetría con la zona de posición, el valor normalizado `lateral` y el desplazamiento en píxeles.

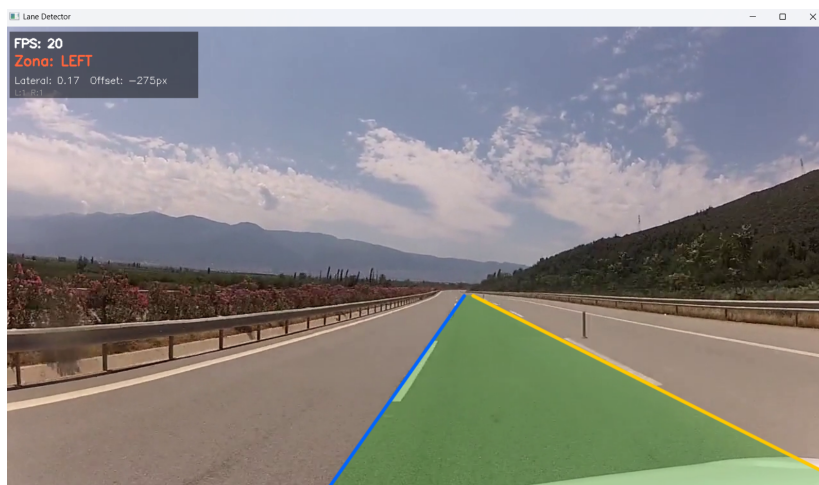


Figura 1: Fotograma de las pruebas previas del detector de carriles sobre vídeo real de carretera, mostrando el polígono de carril, las líneas ajustadas y el panel de telemetría con la zona de posición (`LEFT`) y el desplazamiento lateral.

Los resultados obtenidos sobre vídeo real confirmaron la viabilidad del pipeline de visión clásica antes de su traslado al entorno simulado, validando en particular la robustez del ajuste por regresión lineal y del suavizado temporal frente al ruido visual propio de una escena no controlada (variaciones de iluminación, vegetación lateral y marcas viales parcialmente desgastadas).

3. Control de mantenimiento de carril

Esta sección describe el modelado del vehículo y el proceso de diseño y sintonización del controlador PID de dirección empleado para el mantenimiento de carril. El flujo de diseño sigue la metodología clásica de control: primero se establece un modelo dinámico del sistema, después se sintoniza el controlador en MATLAB/Simulink y, finalmente, los parámetros obtenidos se transfieren directamente a la implementación en Python/ROS2 descrita en la Sección 4.

3.1. Modelado del vehículo

Para el diseño del controlador de dirección se adopta un modelo cinemático de vehículo tipo bicicleta (*bicycle model*), ampliamente utilizado en aplicaciones de control lateral por su equilibrio entre fidelidad y manejabilidad analítica. En este modelo el vehículo queda descrito por su posición lateral y , su ángulo de guiñada ψ y el ángulo de dirección de las ruedas delanteras δ .

El error de seguimiento de carril se define como el desplazamiento lateral del punto central del eje delantero respecto al centro del carril, calculado a partir de la imagen de la cámara frontal tal como se describe en la Sección 2. Este error, expresado en metros, es la señal de referencia del bucle de control de dirección.

El modelo del vehículo simulado en MATLAB/Simulink toma como referencia los parámetros dinámicos del vehículo disponible en el entorno de simulación, siguiendo el mismo esquema que el ego-vehicle empleado en CARLA (Tesla Model 3). Los parámetros relevantes del modelo son la distancia entre ejes (*wheelbase*) L y la velocidad de avance v , que se trata como perturbación paramétrica durante la sintonización dado que el controlador de cruce la regula de forma independiente (véase la Sección 4).

3.2. Estructura de control en cascada

El sistema de control se organiza en dos lazos independientes que operan en cascada:

1. **Lazo externo — control de velocidad (PI):** regula la velocidad longitudinal del vehículo en torno a la referencia deseada v_{ref} , actuando sobre el acelerador y el freno.
2. **Lazo interno — control de dirección (PID):** emplea el error lateral e_s proporcionado por el módulo de detección de carriles para calcular el ángulo de volante corrector y mantener el vehículo centrado en el carril.

Esta arquitectura en cascada permite diseñar y sintonizar cada lazo de forma independiente, aprovechando la separación de escalas temporales entre la dinámica longitudinal

(lenta) y la lateral (más rápida).

3.3. Sintonización en MATLAB/Simulink

El proceso de sintonización se realiza íntegramente en MATLAB/Simulink antes de trasladar los parámetros al entorno ROS2/CARLA. El diagrama de bloques del modelo incluye: el generador de error lateral de referencia, el bloque PID de dirección, el modelo cinemático del vehículo y el bloque de realimentación que cierra el lazo.

Para evaluar la respuesta del controlador se inyecta un error lateral artificial en el punto de entrada del PID. Esta estrategia permite explorar la respuesta transitoria del sistema ante perturbaciones de posición sin necesidad de una trayectoria de referencia compleja, facilitando el ajuste iterativo de las ganancias.

Las ganancias se sintonizaron siguiendo un proceso manual asistido por la herramienta `pidTuner` de MATLAB, imponiendo los siguientes criterios de diseño:

- Tiempo de establecimiento reducido: el vehículo debe recuperar el centro del carril en el menor número de muestras posible.
- Sobreoscilación acotada: se tolera una sobreoscilación pequeña para no sacrificar velocidad de respuesta.
- Error estacionario nulo: el término integral elimina cualquier desviación permanente debida a perturbaciones constantes (p. ej., peralte de la calzada).
- Rechazo de oscilaciones: el término derivativo amortigua las oscilaciones producidas por la dinámica lateral del vehículo.

La Figura 2 muestra la respuesta del sistema ante un error lateral artificial obtenida en Simulink. El error converge al valor final de $-0,006$ m, con oscilaciones amortiguadas y sin sobreoscilación excesiva, lo que valida la sintonización antes de pasar al entorno real de simulación.

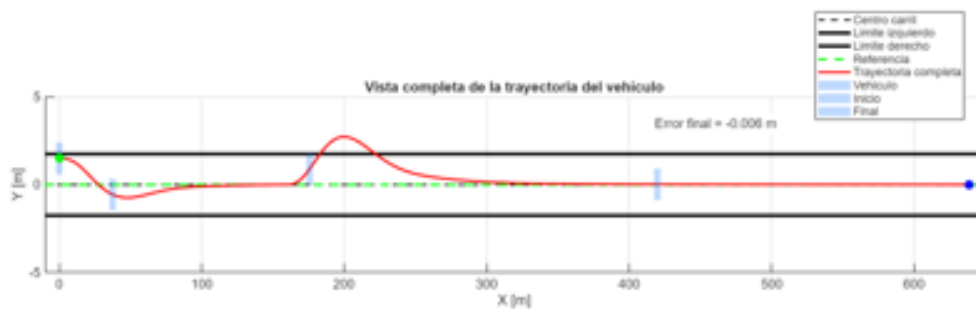


Figura 2: Respuesta del sistema de control de dirección ante un error lateral artificial simulado en MATLAB/Simulink. Error final: $-0,006$ m. Se aprecia la convergencia al centro del carril con oscilaciones amortiguadas.

3.4. Parámetros del controlador

La Tabla 1 recoge los parámetros definitivos de ambos PID's resultantes del proceso de sintonización. Estos valores se configuran directamente en el fichero `params.yaml` del

paquete ROS2 y se cargan en tiempo de ejecución por el nodo `vehicle_control_node`.

Cuadro 1: Parámetros del controlador PID del sistema.

Controlador	Variable controlada	K_p	K_i	K_d
PID de dirección	Error lateral (m)	$2,00 \times 10^{-3}$	$1,00 \times 10^{-3}$	$1,07 \times 10^{-5}$

3.5. Transferibilidad de las ganancias

Una de las hipótesis de partida del trabajo es que las ganancias sintonizadas en MATLAB/Simulink son directamente transferibles a la implementación en Python/ROS2 sin necesidad de reajuste. Esta hipótesis se verifica experimentalmente en la Sección 4: el sistema opera correctamente a las velocidades de 18 km/h y 30 km/h con los mismos valores de K_p , K_i y K_d obtenidos en simulación, lo que confirma la validez del modelo utilizado para la sintonización y la coherencia entre el entorno MATLAB/Simulink y el simulador CARLA.

4. Integración en simulación y resultados

Esta sección describe cómo los módulos de detección de carriles y control PID se integran en un sistema completo que opera en simulación en tiempo real. Se presentan el entorno de simulación empleado, la arquitectura software basada en ROS2, la infraestructura de contenedores Docker, los nodos que componen el sistema, la herramienta de monitorización y los resultados experimentales obtenidos.

4.1. Entorno de simulación: CARLA Simulator

Para el desarrollo y validación del sistema se ha empleado *CARLA Simulator* (*Car Learning to Act*), un simulador de conducción autónoma de código abierto basado en el motor gráfico *Unreal Engine 4*. CARLA proporciona sensores virtuales de alta fidelidad —cámaras RGB, cámaras de segmentación semántica y medidores de velocidad— que permiten diseñar, depurar y validar algoritmos de percepción y control sin necesidad de un vehículo real.

El simulador se ejecuta en su versión *0.9.15* dentro de un contenedor Docker con acceso a la GPU de la máquina anfitriona. Las principales características del entorno configurado son:

- Mapa: Town04, escenario de carretera inter-urbana con marcas viales bien definidas.
- Vehículo (*ego-vehicle*): modelo basado en el Tesla Model 3, controlado en lazo cerrado desde ROS2.
- Sensores disponibles:
 - Cámara RGB frontal (`rgb_front`): imagen principal usada para la detección de carriles.

- Cámara de segmentación semántica (`semantic_segmentation_front`): proporciona la máscara de *ground truth* utilizada en la generación del dataset sintético.
- Medidor de velocidad (`speedometer`): velocidad actual del vehículo en m/s.
- Cámara exterior (`rgb_view`): vista en tercera persona para supervisión visual.

La Figura 3 muestra las dos perspectivas principales empleadas durante las sesiones de conducción autónoma.



(a) Vista exterior (tercera persona).



(b) Cámara frontal del vehículo.

Figura 3: Perspectivas en CARLA Simulator.

4.2. Arquitectura del sistema

El sistema completo se organiza en tres capas que se comunican a través de una red Docker en modo *host*:

1. CARLA Simulator: simulador en su propio contenedor Docker con acceso a la GPU, que publica imágenes y telemetría como tópicos ROS2.
2. CARLA-ROS2 Bridge: puente oficial (`carla_ros_bridge`) que traduce los datos de los sensores de CARLA en mensajes ROS2 estándar y convierte los comandos de control ROS2 en actuaciones sobre el vehículo simulado.
3. Paquete de detección y control (`deteccion_seguimiento_carril`): paquete ROS2 propio que aloja los nodos de detección de carriles, control del vehículo y generación de anotaciones visuales.

El flujo de datos es el siguiente: CARLA publica las imágenes de la cámara frontal en `/carla/ego_vehicle/rgb_front/image` y la velocidad en `/carla/ego_vehicle/speedometer`. El `lane_detection_node` procesa cada *frame* y publica el error lateral en metros en `/lane_detection/lane_error` y el estado completo del carril como JSON en `/lane_detection/lane_state`. El `vehicle_control_node` consume el error lateral y la velocidad para calcular y publicar los comandos de actuación en `/carla/ego_vehicle/vehicle_control_cmd`. Por último, el `annotation_generator_node` recoge el estado de todos los nodos y genera una capa

de anotaciones visuales en `/foxglove/annotations`, que Foxglove Studio superpone sobre la imagen en tiempo real.

La Figura 4 muestra el diagrama de bloques completo del sistema.

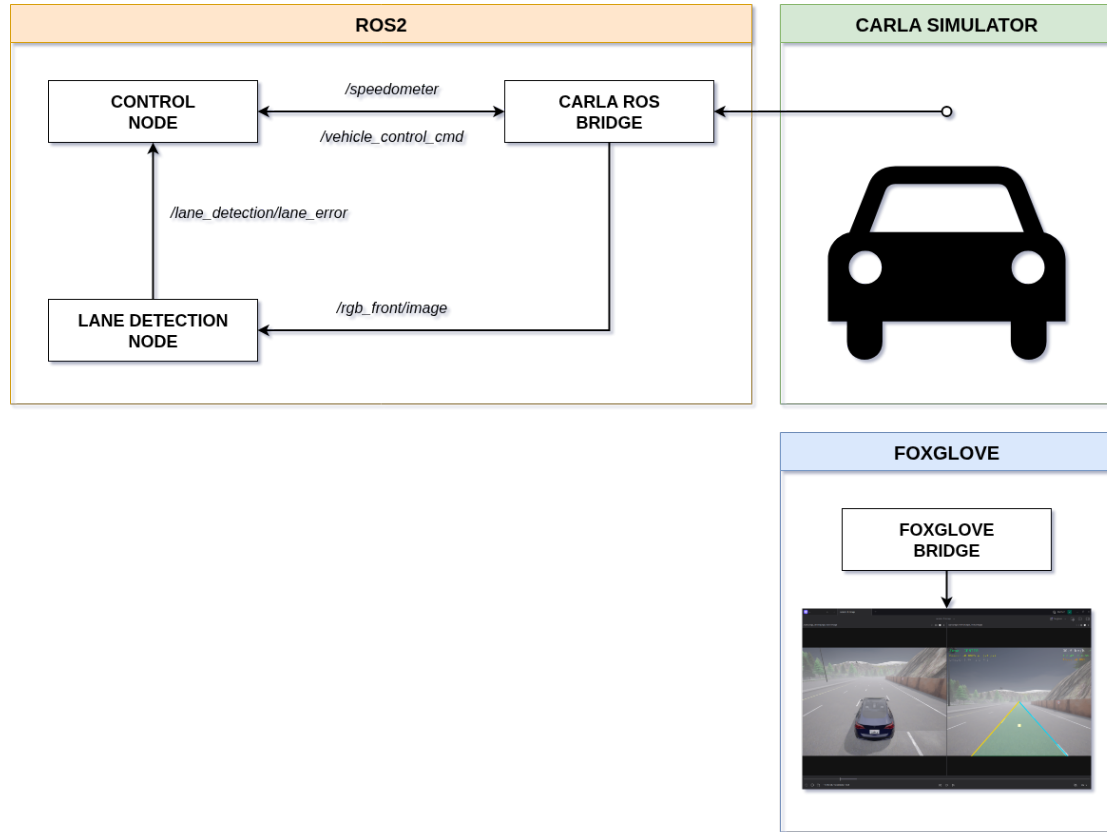


Figura 4: Diagrama de la arquitectura del sistema.

4.3. Infraestructura: Docker y contenedores

Todo el sistema se orquesta mediante **Docker Compose**, levantando los tres servicios con un único comando desde la raíz del repositorio:

```
1 xhost +local:docker
2 docker compose -f tools/docker-compose.yaml up
```

La siguiente tabla describe cada uno de los servicios:

Servicio	Contenedor	Función
ros_bridge	carla_ros_bridge	Bridge CARLA→ROS2
navegacion	deteccion_seguimiento_carril	Lanzamiento del paquete de ROS2
foxglove	foxglove_bridge	WebSocket bridge para Foxglove Studio

La imagen Docker base es `ros:humble-ros-base-jammy`, sobre la que se instalan ROS2 Humble Hawksbill, las librerías de visión artificial (`python3-opencv`, `ros-humble-cv-bridge`),

las herramientas de visualización (`ros-humble-foxglove-bridge`, `ros-humble-foxglove-msgs`) y el cliente Python de CARLA 0.9.15 junto con el `carla-ros-bridge` compilado desde el código fuente.

El código fuente del paquete ROS2 se monta como volumen dentro del contenedor de navegación. Esto permite que cualquier cambio en el host quede disponible tras ejecutar `colcon build -symlink-install` sin necesidad de reconstruir la imagen Docker.

4.4. Nodos ROS2 del sistema

El paquete `deteccion_seguimiento_carril` implementa tres nodos ROS2 en Python que se lanzan de forma conjunta mediante `run.launch.py`, más un nodo adicional de recolecta de datos que se activa por separado.

4.4.1. Nodo de detección de carriles (`lane_detection_node`)

Este nodo recibe cada *frame* de la cámara frontal y ejecuta el pipeline de visión clásica descrito en la Sección 2. A partir de las líneas detectadas calcula el error lateral del vehículo respecto al centro del carril: primero en píxeles (`offset_px`) y luego en metros, dividiendo por la longitud focal f_x extraída del mensaje `camera_info`. Los resultados se publican en:

- `/lane_detection/lane_error` (`std_msgs/Float32`): error lateral en metros.
- `/lane_detection/lane_state` (`std_msgs/String`): JSON con las coordenadas de las líneas izquierda y derecha, la zona de posición (CENTER/LEFT/RIGHT) y los flags de detección.

4.4.2. Nodo de control del vehículo (`vehicle_control_node`)

Este nodo cierra el lazo de control: recibe el error lateral y la velocidad actual, y genera los comandos de actuación sobre el vehículo. Opera a 20 Hz e implementa dos controladores PID independientes.

PID de velocidad (control de crucero). Regula la velocidad del vehículo en torno a la referencia configurable `target_speed`. El error de control es la diferencia entre la velocidad objetivo y la medida por el velocímetro. La salida del PID se mapea a los canales *throttle* (acelerador) y *brake* (freno) normalizados en $[0, 1]$:

$$e_v(t) = v_{\text{ref}} - v_{\text{actual}}$$

$$u_v(t) = K_{p,v} e_v + K_{i,v} \int e_v dt$$

$$\text{throttle} = \max(0, \min(1, u_v)), \quad \text{brake} = \max(0, \min(1, -u_v))$$

PID de dirección (mantenimiento de carril). Ajusta el ángulo del volante para mantener el vehículo centrado en el carril. El PID opera en radianes y su salida se normaliza por el ángulo físico máximo del volante ($\theta_{\text{máx}} = 0,6 \text{ rad} \approx 34^\circ$) para obtener

el valor de dirección normalizado de CARLA en $[-1, 1]$. La convención de signo es: error positivo (vehículo desplazado a la derecha) produce una corrección hacia la izquierda ($\text{steer} < 0$):

$$u_s(t) = K_{p,s} e_s + K_{i,s} \int e_s dt + K_{d,s} \dot{e}_s$$

$$\text{steer} = \max\left(-1, \min\left(1, \frac{u_s}{\theta_{\text{máx}}}\right)\right)$$

Ambos PIDs incorporan *anti-windup* que limita el término integral al rango $[-10, 10]$. Los parámetros definitivos configurados en `params.yaml` se recogen en la Tabla 1. Estos valores son el resultado de la sintonización en MATLAB/Simulink descrita en la Sección 3, trasladados directamente a la implementación en Python.

4.4.3. Nodo generador de anotaciones (`annotation_generator_node`)

Este nodo actúa como capa de visualización del sistema. En cada *frame* de la cámara frontal recoge el estado de los demás nodos y genera un mensaje `foxglove_msgs/ImageAnnotations` que Foxglove Studio superpone sobre la imagen en tiempo real. Las anotaciones incluyen:

- Polígono de carril: región semitransparente en verde que cubre el área delimitada por las líneas izquierda y derecha detectadas.
- Líneas de carril: línea izquierda en amarillo y derecha en cian, con grosor de 5 px.
- Vector de desviación: segmento desde el centro geométrico de la imagen hasta el centro estimado del carril, en amarillo.
- HUD de telemetría: zona del carril (`CENTER/LEFT/RIGHT`), error lateral en metros y en píxeles, velocidad actual en km/h, y valores de *throttle*, *brake* y *steer*.

La Figura 5 muestra la interfaz de Foxglove Studio durante una sesión de conducción.



Figura 5: Interfaz de Foxglove Studio durante una sesión de conducción autónoma. A la izquierda, la vista exterior del vehículo; a la derecha, la cámara frontal con las anotaciones superpuestas: líneas del carril, polígono de área y HUD de telemetría.

4.5. Generación de datasets sintéticos

Una de las ventajas de trabajar con un simulador como CARLA es la posibilidad de generar datos etiquetados de forma automática. La cámara de segmentación semántica asigna a cada píxel una etiqueta de clase con precisión perfecta, ofreciendo un *ground truth* ideal para evaluar el detector de carriles sin ningún coste de anotación manual.

El nodo `lane_dataset_node` opera en modo conducción manual (sin el nodo de control) y captura imágenes a 5 Hz. Para cada instante capturado:

1. Obtiene la máscara semántica del mismo instante de tiempo.
2. Detecta los píxeles de marcas viales en la máscara y los proyecta sobre la imagen RGB.
3. Almacena el par imagen RGB / máscara binaria en `dataset/lanes/images/` y `dataset/lanes/masks/` respectivamente.

La recolecta se lanza con:

```
1 xhost +local:docker
2 docker compose -f tools/docker-compose-lane-dataset.yaml up
```

El repositorio incluye además la herramienta `tools/visualize_lanes_dataset.py` para revisar el dataset generado y `tools/make_lane_video.py` para producir un vídeo con el detector de carriles aplicado sobre las imágenes del dataset.

4.6. Resultados

Las pruebas de conducción autónoma se han realizado a dos velocidades de referencia: 18 km/h (5.0 m/s) y 30 km/h (8.33 m/s), modificando únicamente el parámetro `target_speed` en `params.yaml`. En ambos casos el sistema es capaz de mantener el vehículo centrado en el carril de forma continuada sin intervención humana.

Los vídeos de las sesiones de conducción autónoma a 18 km/h y 30 km/h, así como un vídeo de demostración del detector de carriles sobre el dataset sintético, están disponibles en el repositorio público de GitHub:

<https://github.com/JoseLopez36/Deteccion-Seguimiento-Carril>

Los ficheros correspondientes son `media/Demo_18_kmh.mp4`, `media/Demo_30_kmh.mp4` y `media/Deteccion_Carriles.mp4`.

5. Conclusiones

Este trabajo ha permitido implementar un sistema de *Lane Keeping Assist* funcional de extremo a extremo, demostrando que la combinación de técnicas de visión clásica con un controlador PID en cascada y una arquitectura ROS2 modular constituye una solución eficaz y fácilmente reproducible para la conducción autónoma en simulación.